⚡ Transformer Deep Learning Universal Target Tracking (Super Tracker)

1. Course Overview

The function implemented in this lesson can be simply understood as: **mouse box-select any object, real-time inference based on Transformer deep learning model, robotic arm tracks precisely and smoothly like radar**.

Unlike traditional tracking algorithms based on color threshold segmentation or manual feature filtering (such as KCF), this case adopts a high-performance general-purpose single-target tracking algorithm (Super Tracker) based on **Transformer deep neural network**. The algorithm uses onboard GPU/TensorRT engine for hardware-accelerated inference, providing excellent resistance to light/shadow mutations, deformation, and background clutter interference for any unknown object (whether wooden blocks, faces, hands, dolls, or personal items). The robotic arm uses visual servo adaptive closed-loop control for base rotation and arm pitching, keeping the tracked object always locked in the center of the visual field.

> **Note**
>
> **This lesson uses deep learning universal tracking, no tags or color input needed, supports box-selecting any natural object in the field of view.**

---

2. Preparation and Running

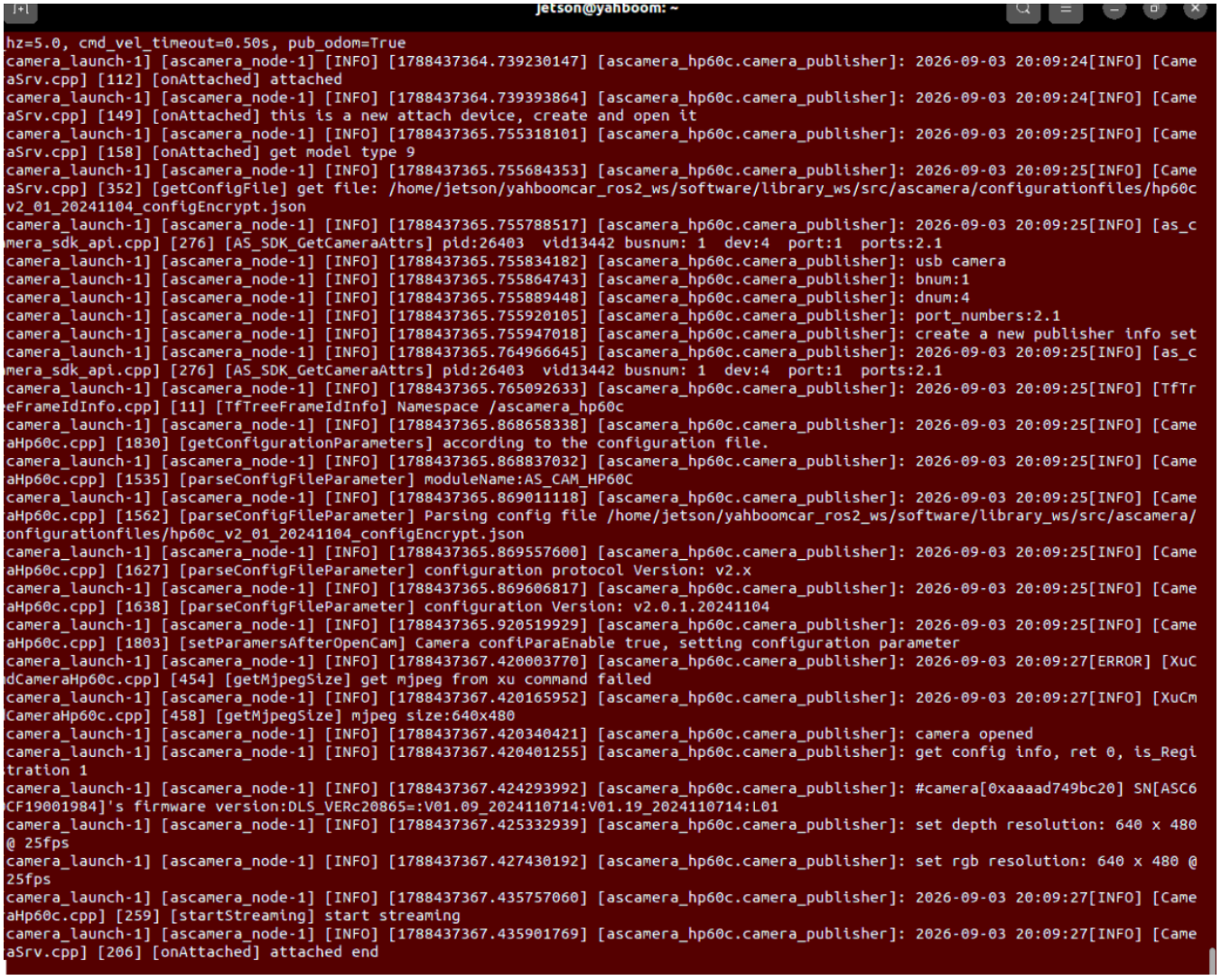
This case requires opening 3 terminals to start the system sequentially:

2.1 Start Camera Driver and Low-Level Robotic Arm Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

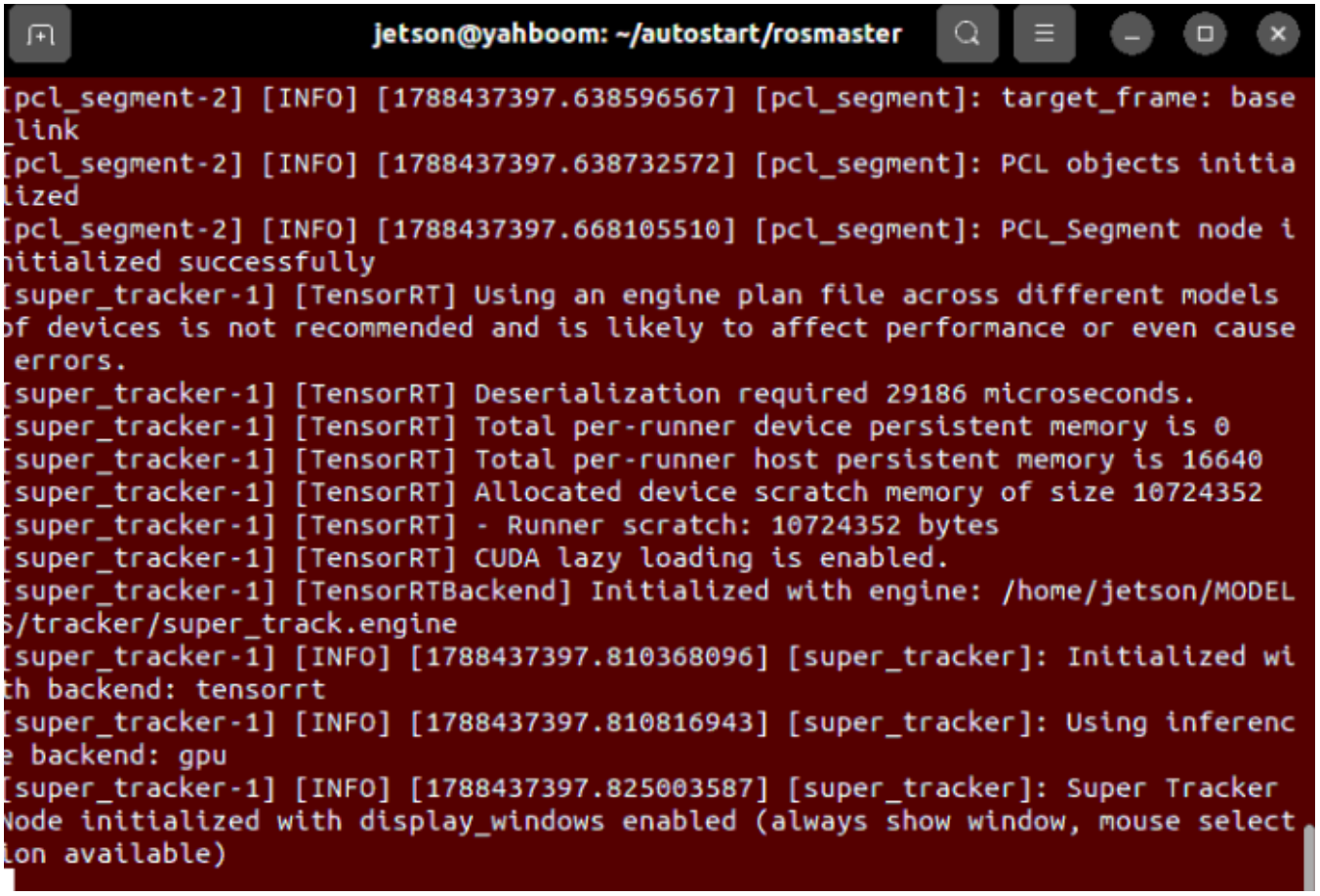
This command initializes the vehicle chassis and robotic arm bus driver communication, starts camera image publishing service, and loads the kinematics solver environment.



2.2 Start Super Tracker Deep Learning Tracking System

Run in **Terminal 2**:

```
ros2 launch dofbot_pro_bringup super_tracker.launch.py
```



> **Tip**
>
> The startup script has built-in camera type auto-detection logic (`resolve_camera_type()`). If you need to explicitly specify the camera model, you can add parameters:
>
> - With **3D Depth Camera (Nuwa)**: `ros2 launch dofbot_pro_bringup super_tracker.launch.py camera_type:=nuwa`
> - With **USB Monocular Camera**: `ros2 launch dofbot_pro_bringup super_tracker.launch.py camera_type:=usb`

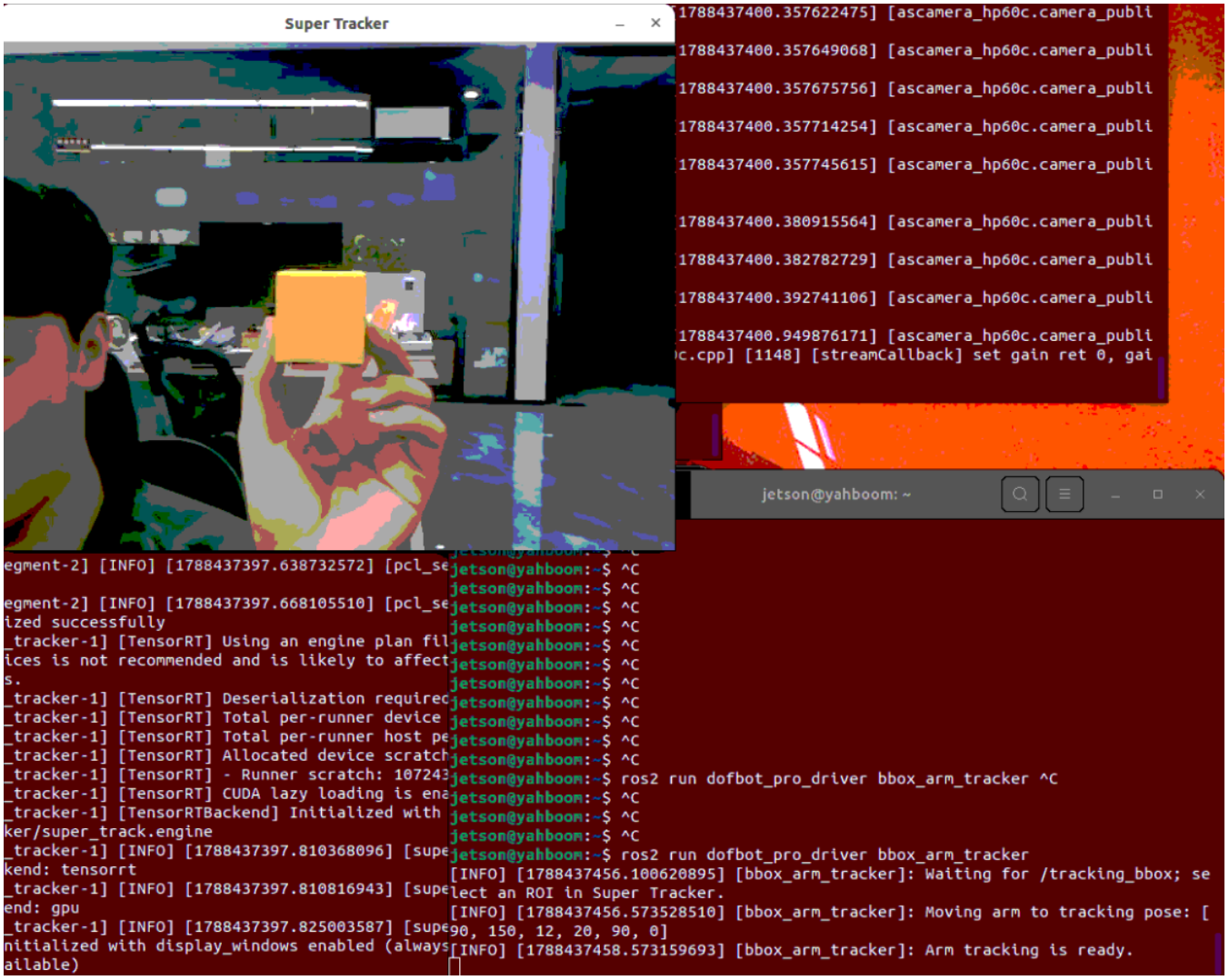
This Launch node automatically loads the TensorRT neural network inference model (`super_track.engine`) saved in the system, and pops up a visualization tracking window on screen. When configured with a 3D depth camera, the system also links to start the point cloud segmentation node (`pcl_segment`).

2.3 Start Robotic Arm Visual Servo Following Control Node

Run in **Terminal 3**:

```
ros2 run dofbot_pro_driver bbox_arm_tracker
```

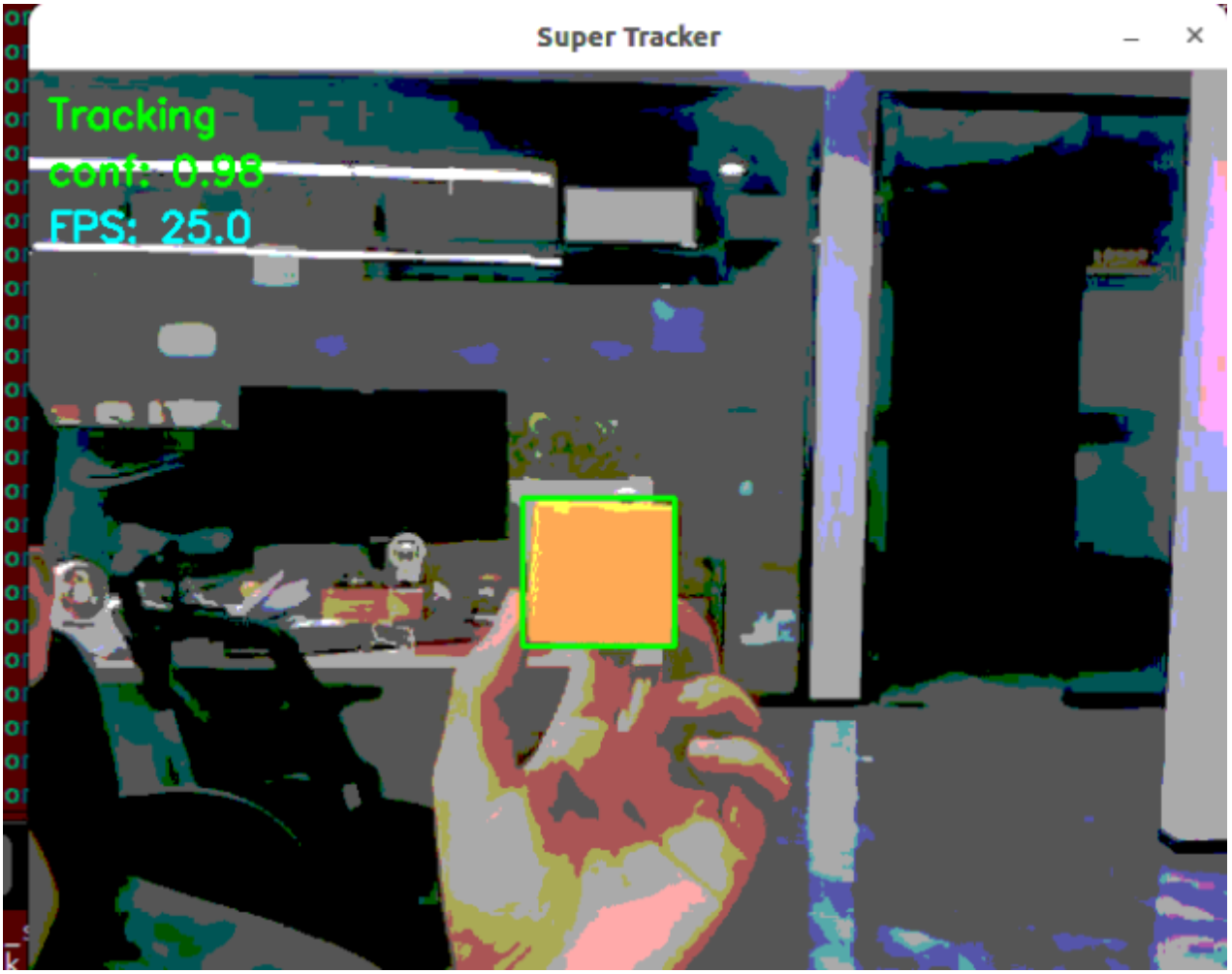
This node subscribes to the target bounding box topic `/tracking_bbox`, real-time computes pixel center and image optical center dynamic error, and through adaptive multi-level PID control of robotic arm joints achieves smooth, jitter-free tracking.



3. Operation Demo and Interaction Flow

1. **Robotic arm forward-looking positioning**:
    - After node starts, the robotic arm automatically and smoothly rises to the forward-looking wide-range patrol pose: `[90°, 150°, 12°, 20°, 90°, 0°]`;
2. **Mouse box-select target**:
    - In the popped up Super Tracker image window, **press C then hold left mouse button and drag** to draw a rectangle, box the target object to track;
    - Release left mouse button, press Enter;
3. **Omnidirectional dynamic following in space**:
    - Slowly move the target object horizontally or up/down in front of the robotic arm:
        - **Target moves left/right**: Robotic arm joint 1 base servo rotates in real-time for tracking, always keeping the target at the horizontal center of the image;
        - **Target moves up/down**: Robotic arm joint 4 wrist pitching joint finely adjusts; when the target is at an extremely low position causing joint 4 downward pitch to hit the limit (arriving at limit angle), joint 2 large arm will automatically coordinate downward pitch, expanding the vertical working blind zone;
4. **Occlusion protection and reselection**:

- If the target is occluded or quickly moves out of view, the robotic arm automatically maintains the current pose on standby (won't shake disorderly);
- Once the target re-enters the view, tracking smoothly resumes; press `C` key in the image window to reset and box-select a new object at any time;



4. Core Source Code Analysis

The intelligent visual tracking system is completed by deep learning inference end and multi-axis motion control end working in coordination:

Module Name	Source Code Path	Core Function
Inference Startup Master	`dofbot_pro_bringup/launch/super_tracker.launch.py`	Hardware camera mode detection, TensorRT engine accelerated loading, point cloud linking
Robotic Arm Servo Following	`dofbot_pro_driver/dofbot_pro_driver/bbox_arm_tracker.py`	EMA low-pass smoothing, deadband filtering, multi-level joint motion mapping and limiting

4.1 Launch Startup and Hardware Multi-Mode Adaptive (`super_tracker.launch.py`)

The Launch script implements seamless dynamic switching between USB monocular camera and Nuwa depth camera through Python expressions, with GPU hardware inference acceleration as the core driver:

```
# super_tracker.launch.py —— Camera adaptive and GPU model loading
camera_type = LaunchConfiguration('camera_type')

# Automatically remap image topic based on camera type
image_topic = PythonExpression([
    "'/usb_cam/image_raw' if '", camera_type, "' == 'usb' else "
    "'/ascamera_hp60c/camera_publisher/rgb0/image'"
])

# Load high-performance TensorRT model engine
super_track_model = os.path.join(
    os.path.expanduser('~'), 'MODELS', 'tracker', 'super_track.engine'
)

# Only activate point cloud ROI segmentation node when equipped with 3D depth camera
pclsegment_node = Node(
    package='smart_tracker',
    executable='pcl_segment',
    condition=LaunchConfigurationEquals('camera_type', 'nuwa'),
    ...
)
```

4.2 First-Order Low-Pass Filtering and Tracking Deadband Anti-Jitter (`bbox_arm_tracker.py`)

The target detection output pixel center often has tiny single-frame jitter. The program introduces first-order exponential moving average (EMA) smoothing filter and pixel deadband mechanism to completely prevent the robotic arm from producing resonant micro-vibration:

```
# bbox_arm_tracker.py —— Exponential smoothing and deadband control
def _bbox_callback(self, message):
    center_x = (x1 + x2) * 0.5
    center_y = (y1 + y2) * 0.5

    # Exponential moving average smoothing filter (alpha = 0.35)
    alpha = self.smoothing_alpha
    self._center_x += alpha * (center_x - self._center_x)
    self._center_y += alpha * (center_y - self._center_y)

def _bounded_step(self, error, deadband):
    # When error is less than deadband (±15 pixels), output 0 to avoid frequent micro-
movements
    if abs(error) <= deadband:
```

```
        return 0.0
    magnitude = max(1.0, (abs(error) - deadband) * self.proportional_gain)
    # Single cycle maximum step limiting (max_step_deg = 2.0°)
    return math.copysign(min(self.max_step_deg, magnitude), error)
```

4.3 Multi-Joint Multi-Level Coordinated Kinematics Control

For pitch control, the program designs an elegant **multi-level cascade adjustment mechanism**: prioritize using joint 4, which has the fastest dynamic response and smallest inertia, for fine adjustment; when joint 4 reaches its lower limit but the target is still below, joint 2 large arm coordinates downward press, greatly expanding the robotic arm's vertical working field of view:

```
# bbox_arm_tracker.py —— Joint multi-level coordinated computation
if y_step != 0.0:
    # Prioritize adjusting joint 4 wrist pitch joint
    next_joint4 = self._clamp(
        next_joints[3] + self.joint4_direction * y_step,
        self.joint4_min, self.joint4_max
    )
    joint4_changed = abs(next_joint4 - next_joints[3]) >= 0.5
    next_joints[3] = next_joint4

    # When joint 4 reaches lower limit and target is still below, cascade drive joint 2
large arm to coordinate downward pitch
    if not joint4_changed and error_y > self.deadband_y:
        next_joints[1] = self._clamp(
            next_joints[1] + self.joint2_down_direction * abs(y_step),
            self.joint2_min, self.joint2_max
        )
```

4.4 Watchdog Safety Protection Mechanism

To ensure physical mechanism safety and reliability, the node configures strict timeout watchdog and confidence threshold:

1. **Confidence threshold** (`confidence_threshold = 0.08`): When confidence is too low, judged as invalid candidate target;
2. **Timeout protection** (`bbox_timeout_sec = 0.5s`): If no updated data is received for more than 0.5 seconds, immediately freeze the robotic arm's current pose, keep standby state, preventing divergence and loss of control after tracking target disappears.